

PERSONALIZED NETWORKING ASSISTANT

Team / Student Details :

- Team Member: D.Satya Kalyani
- Team Member: N.Sravanthi
- Team Member: P.Baby Naga Lakshmi
- Team Member: M.Venkata Kowshik
- Team Member: K.Asha Jyothi

Introduction:

The Personalized Networking Assistant is an AI-powered web application designed to help users generate smart, tailored conversation starters for professional or social networking events. It leverages advanced Natural Language Processing (NLP) models — **DistilBERT** for theme extraction from event descriptions and **GPT-2** for generating context-aware conversation prompts. The platform also includes fact-checking capabilities using Wikipedia APIs and maintains a conversation history with user feedback for continuous personalization and improvement.

Description:

The Personalized Networking Assistant is a full-stack AI-powered web application that assists users in preparing for networking events by generating intelligent, personalized conversation starters. Built with a responsive and interactive design, the application uses **DistilBERT** for zero-shot theme classification from event descriptions and user bios, and **GPT-2** for generating relevant, context-aware prompts.

Additional features include real-time fact verification via Wikipedia API, conversation history logging, and a feedback mechanism (thumbs up/down) to refine future suggestions. The backend is powered by **FastAPI** for high-performance RESTful services, while the frontend uses **Streamlit** for a clean, Python-native interactive UI. The entire system emphasizes modularity, testability (with `pytest`), and seamless integration between AI components.

Scenario-Based Case Studies:

Scenario 1: Generating Smart Starters

A user enters an event description such as *"AI for Sustainable Cities"* and specifies interests like *"climate change"* and *"urban planning."* The assistant extracts key themes (e.g., AI, sustainability, urban development) using DistilBERT and generates 2–3 tailored conversation starters aligned with the user's goals and profile.

Scenario 2: Quick Fact Verification

While preparing for a conference, a user requests a quick fact-check on *"blockchain in healthcare."* The application queries the Wikipedia API, summarizes reliable information, and presents verified key points to help the user sound knowledgeable during discussions.

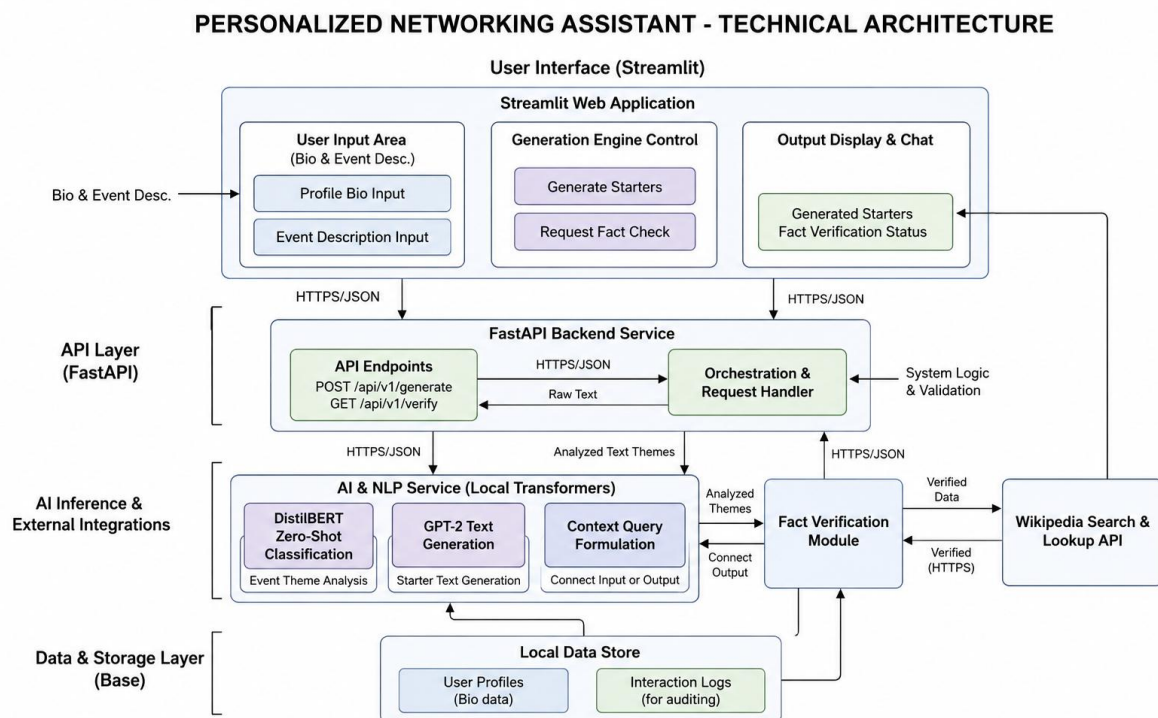
Scenario 3: Reviewing Past Strategies

After attending multiple events, a user accesses the *"History"* section to review previously generated conversation starters, see which ones received positive feedback (thumbs up), and analyze patterns in successful interactions. This feature supports continuous improvement in the user's networking approach.

Technical Architecture:

The application follows a **modular client-server architecture**:

- **Frontend:** Streamlit-based responsive web interface for user input (bio, event description, interests), controls (Generate / Fact Check), and output display (starters + chat-like interface).
- **Backend:** FastAPI service handling API endpoints (/generate, /verify, etc.), request orchestration, and integration with AI models.
- **AI & NLP Layer:**
 - DistilBERT (zero-shot classification) for theme extraction.
 - GPT-2 for text generation of conversation starters.
 - Context formulation and Wikipedia API for fact verification.
- **Data Layer:** Local storage for user profiles (bio data) and interaction logs (history + feedback) for personalization and auditing.
- **Communication:** HTTPS/JSON between layers for secure and efficient data exchange.

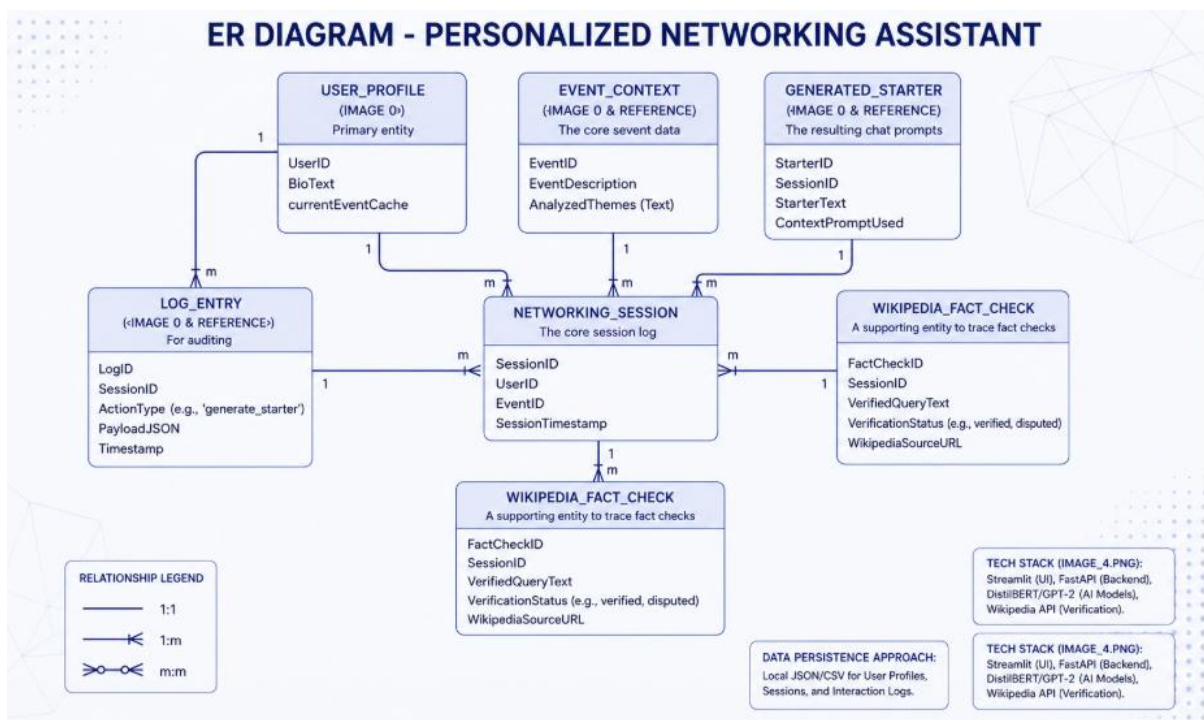


Key Features:

- **Smart Conversation Starter Generation** — Context-aware prompts using DistilBERT + GPT-2

- **Theme Extraction & Personalization** — Analyzes user bio and event details.
- **Fact Verification Module** — Real-time reliable summaries from Wikipedia API.
- **Conversation History & Feedback** — Logs past interactions with thumbs up/down for improvement.
- **Responsive & Interactive UI** — Built with Streamlit for easy use.
- **RESTful APIs** — Clean, documented endpoints with FastAPI.
- **Unit Testing** — Comprehensive tests using pytest.
- **Modular Design** — Easy to maintain and extend.

Data Model – Entity Relationship (ER) Diagram:



ER Diagram Description:

The ER diagram features six primary entities:

- **User Profile** (UserID, BioText, currentEventCache)

- **Event Context** (EventID, EventDescription, AnalyzedThemes)
- **Networking Session** (core session log)
- **Generated Starter**
- **Wikipedia Fact Check**
- **Log Entry** (for auditing)

Key Relationships:

- One user → Many networking sessions (1:M)
- One session → Many generated starters (1:M)
- One session → Many fact checks (1:M)
- One session → Many log entries (1:M)

This normalized structure ensures clear traceability of user data, AI outputs, fact verification, and system logs while reducing redundancy.

Tech Stack:

- **Backend:** FastAPI (Python)
- **Frontend:** Streamlit
- **AI/ML Models:** DistilBERT (Hugging Face Transformers), GPT-2
- **External APIs:** Wikipedia API (for fact-checking)
- **Data Handling:** Local storage / JSON-based logs (can be extended to database)
- **Testing:** pytest
- **Other:** Python 3.x, Hugging Face Transformers library

Pre-requisites:

Before getting started, ensure you have a basic familiarity with cloud computing and version control (Git) will be helpful. Also, install Visual Studio Code for writing and managing code efficiently.

Python 3.11+: Core programming language used to build the application backend, manage logic, handle APIs, and run AI/NLP processing modules.

<https://www.python.org/downloads/>

FastAPI Documentation: High-performance Python web framework used to create REST APIs for communication between the frontend, AI services, and verification modules.

<https://fastapi.tiangolo.com/>

Streamlit Documentation: Interactive frontend framework used to build the web-based user interface for profile input, event descriptions, and displaying generated networking starters.

<https://docs.streamlit.io/>

Hugging Face Transformers: AI/ML library providing pre-trained NLP models such as DistilBERT and GPT-2 for text classification, theme analysis, and starter generation.

<https://huggingface.co/docs/transformers/>

Wikipedia API (Python): External verification service used to fetch and validate factual information from Wikipedia sources in real time.

<https://pypi.org/project/wikipedia/>

Git Version Control: Source code management system used for tracking project changes, collaboration, version history, and deployment workflows.

<https://git-scm.com/doc>

Virtual Environment (venv): Python environment isolation tool used to manage project dependencies separately and maintain consistent package versions across development setups.

<https://docs.python.org/3/library/venv.html>

Roles and Responsibilities:

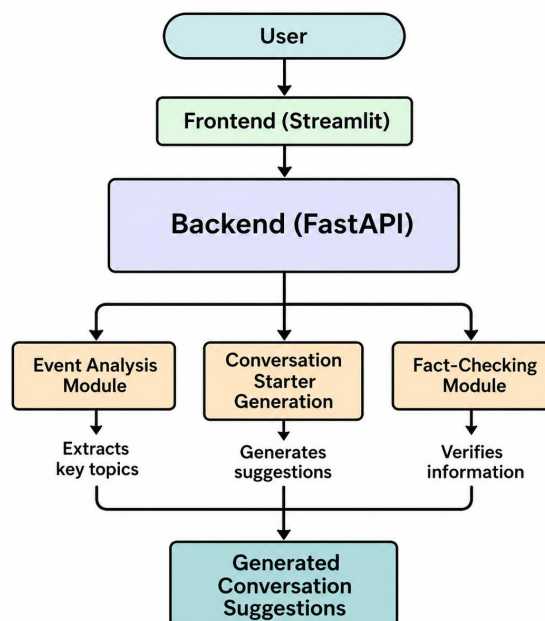
User:

- Provide bio, event description, and interests.
- Generate conversation starters and request fact checks.
- Review history and give feedback (useful/not useful) on suggestions.
- Apply the generated prompts in real networking scenarios.

Developer :

- Designed and implemented the complete architecture.
- Integrated AI models (DistilBERT + GPT-2).
- Developed FastAPI backend services and Streamlit frontend.
- Implemented fact verification and history/feedback system.
- Performed unit testing and ensured modularity.
- Documented the project and prepared for deployment.

Project Flow:



Epic 1: Model Selection and Architecture

- Story 1: Model Research & Selection (DistilBERT + GPT-2)
- Story 2: Defining the Application Architecture
- Story 3: Environment Setup

Epic 2: Core Functionalities Development

- Story 1: Data Schema Definition
- Story 2: Event Analyzer Service Development
- Story 3: Topic Generator Service Development
- Story 4: Fact Checker Service Development
- Story 5: History Logger Service Development
- Story 6: Feedback Logger Service Development
- Story 7: API Routes Implementation
- Story 8: Application Entry Point Configuration

Epic 3: Main Application Logic

- Story 1: Centralized Routing Architecture
- Story 2: Service Layer Design Principles
- Story 3: FastAPI-Specific Features Integration

Epic 4: Frontend UI Using Streamlit

- Story 1: Application Setup and Configuration
- Story 2: Input Section and Main Generation Flow
- Story 3: Results Display and Feedback System
- Story 4: Fact-Checking Section
- Story 5: Conversation History View

- Story 6: Feedback History View
- Story 7: Streamlit State Management Implementation

Epic 5: Testing and Local Deployment

- Story 1: Testing Philosophy and Framework Selection (pytest)
- Story 2–4: Service Testing (Event Analyzer, Topic Generator, Fact Checker)
- Story 5: API Routes Testing with httpx TestClient
- Story 6: Running Tests and Interpreting Results
- Story 7: Local Deployment
- Story 8: Manual Testing and Verification

Epic 1: Model Selection and Architecture

Model Research & Selection

The model selection process is arguably the most consequential step in any AI application.

Choosing the wrong model can result in slow responses, poor accuracy, high resource consumption, or output that fails to meet user expectations. For this project, two distinct NLP tasks required model evaluation: event theme classification and conversation text generation.

Theme Extraction: Why DistilBERT Was Chosen

For extracting themes from event descriptions, several Hugging Face transformer models were evaluated. The primary requirement was the ability to classify text into categories without task-specific training data — a technique known as **zero-shot classification**. This is critical because the application must work for any type of networking event, not just pre-trained categories.

DistilBERT was ultimately selected because it achieves the optimal balance between inference speed and classification accuracy. For a real-time web application where users expect immediate feedback, a model that responds in milliseconds rather than seconds makes a tangible difference

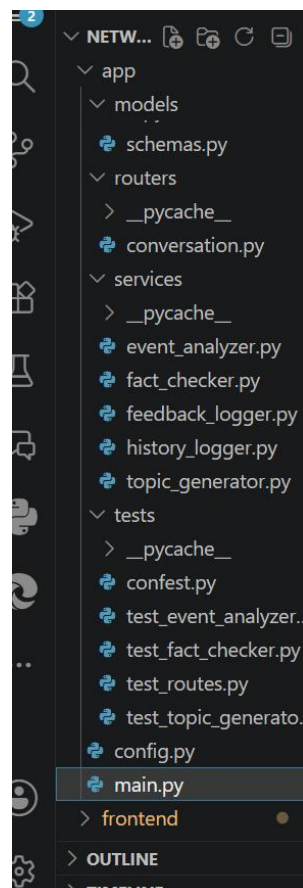
in user experience. Its compact size also makes it far more deployable in resource-constrained environments.

Conversation Generation: Why GPT-2 Small Was Chosen

For generating conversation starters, the requirement was a model capable of producing natural, contextually coherent text based on a structured prompt. Several generative models were evaluated.

GPT-2 Small was selected for its ability to run efficiently on standard hardware without GPU acceleration. While larger models produce higher-quality prose, GPT-2 Small produces conversation starters that are sufficiently natural and engaging for the specific use case — short, punchy opening lines for professional networking. Its integration via the Hugging Face pipeline API also makes it straightforward to configure and deploy.

Defining the Application Architecture



With models selected, the next step was defining a clean, modular system architecture. The architecture needed to support separation of concerns (so different developers or teams can work on frontend, backend, and AI services independently), easy testability, and straightforward extensibility for future features like authentication or database integration.

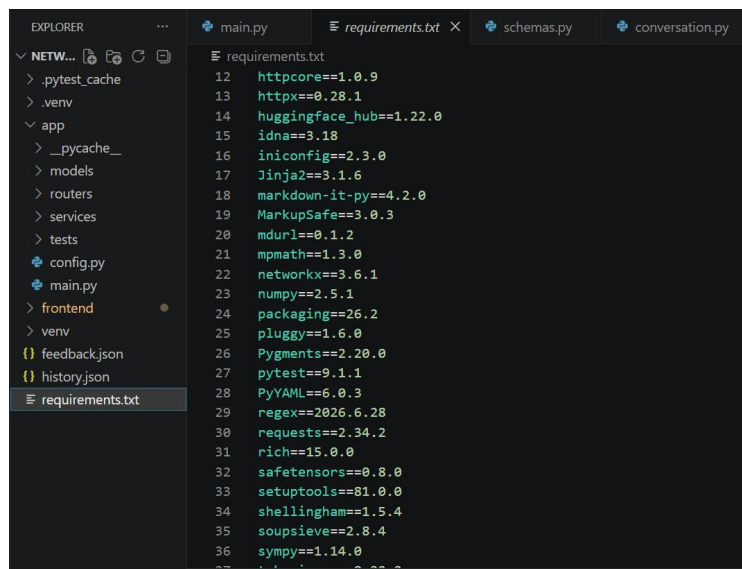
Architectural Overview

The application follows a three-tier architecture with a clear data flow from the user interface through the API layer to the AI service modules:

Directory Structure and Module Design

The project was organized into a clean directory hierarchy that separates application code from frontend, tests, and configuration. This structure makes the codebase navigable and ensures that adding new features requires minimal disruption to existing code:

Environment Setup



The screenshot shows a code editor with a dark theme. On the left, the 'EXPLORER' sidebar displays a directory tree. The tree includes folders like '.pytest_cache', '.venv', 'app', and 'frontend'. The 'requirements.txt' file is selected and highlighted. The main editor area shows the contents of 'requirements.txt', which lists various Python dependencies with their versions, such as 'httpcore==1.0.9', 'httpx==0.28.1', 'huggingface_hub==1.22.0', 'idna==3.18', 'iniconfig==2.3.0', 'Jinja2==3.1.6', 'markdown-it-py==4.2.0', 'MarkupSafe==3.0.3', 'mdurl==0.1.2', 'mpmath==1.3.0', 'networkx==3.6.1', 'numpy==2.5.1', 'packaging==26.2', 'pluggy==1.6.0', 'Pygments==2.20.0', 'pytest==9.1.1', 'PyYAML==6.0.3', 'regex==2026.6.28', 'requests==2.34.2', 'rich==15.0.0', 'safetensors==0.8.0', 'setuptools==81.0.0', 'shellingham==1.5.4', 'soupsieve==2.8.4', and 'sympy==1.14.0'.

```
EXPLORER
├── .pytest_cache
├── .venv
├── app
│   ├── __pycache__
│   ├── models
│   ├── routers
│   ├── services
│   ├── tests
│   ├── config.py
│   ├── main.py
│   ├── frontend
│   ├── venv
│   ├── feedback.json
│   ├── history.json
│   └── requirements.txt
└── requirements.txt
```

```
requirements.txt
12 httpcore==1.0.9
13 httpx==0.28.1
14 huggingface_hub==1.22.0
15 idna==3.18
16 iniconfig==2.3.0
17 Jinja2==3.1.6
18 markdown-it-py==4.2.0
19 MarkupSafe==3.0.3
20 mdurl==0.1.2
21 mpmath==1.3.0
22 networkx==3.6.1
23 numpy==2.5.1
24 packaging==26.2
25 pluggy==1.6.0
26 Pygments==2.20.0
27 pytest==9.1.1
28 PyYAML==6.0.3
29 regex==2026.6.28
30 requests==2.34.2
31 rich==15.0.0
32 safetensors==0.8.0
33 setuptools==81.0.0
34 shellingham==1.5.4
35 soupsieve==2.8.4
36 sympy==1.14.0
```

Consistent environment setup is critical for reproducibility. The team established a Python virtual environment to isolate project dependencies from the system Python installation, preventing version conflicts and ensuring that any developer can recreate the exact same working environment from scratch.

Step-by-Step Environment Configuration

- Create a new Python virtual environment using venv
- Activate the virtual environment (different command on Windows vs Mac/Linux)
- Install all dependencies from the shared requirements.txt file
- Verify installations by importing key packages in a Python shell

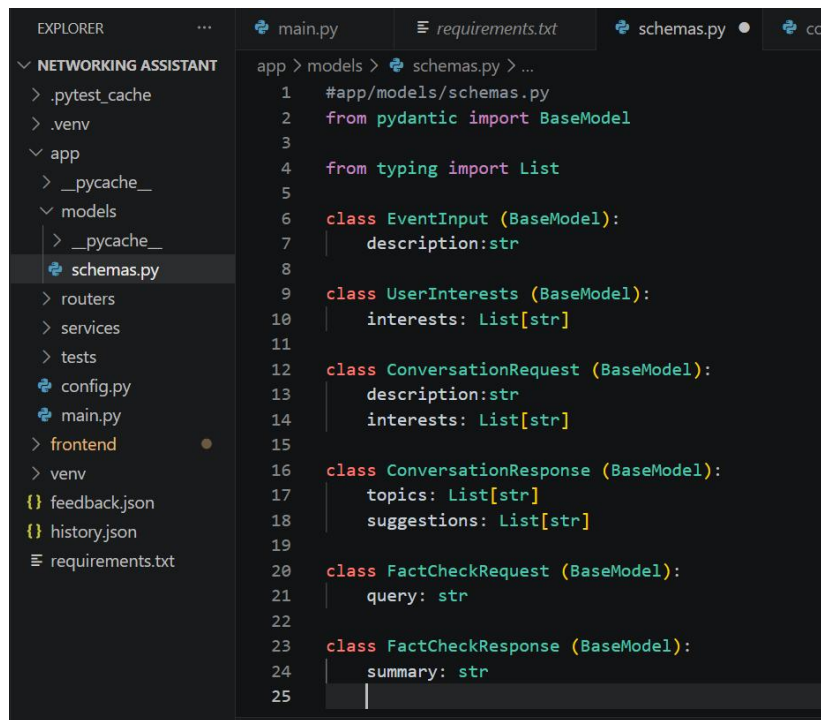
The requirements.txt includes all direct and transitive dependencies needed for the application, pinned to specific versions for reproducibility. Key packages include fastapi, uvicorn, streamlit, transformers, torch, requests, pydantic, pytest, and httpx.

Epic 2: Core Functionalities Development

Data Schema Definition

Before building any service logic or routes, it is essential to define the data contracts between the frontend and backend. These contracts are expressed as Pydantic models, which provide automatic type validation, serialization, and clear documentation through FastAPI's built-in Swagger UI.

This file defines six Pydantic model classes that form the backbone of all data exchange in the application. By inheriting from BaseModel, each class gains automatic validation — if a required field is missing or has the wrong type, FastAPI will automatically return a 422 Unprocessable Entity error with a clear description of what went wrong, without requiring any additional error-handling code

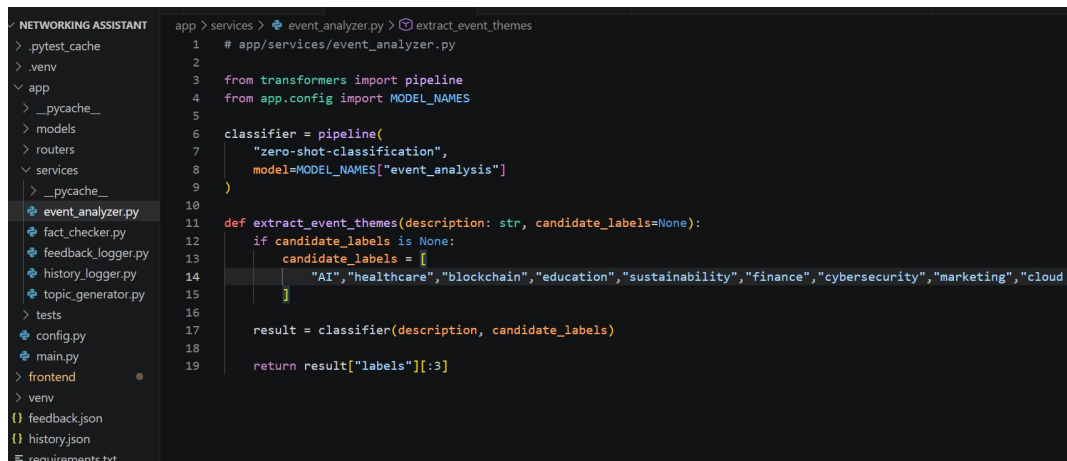


The screenshot shows a code editor with a dark theme. On the left is the 'EXPLORER' sidebar showing a file tree for a project named 'app'. The tree includes folders like '.pytest_cache', '.venv', and 'models', and files like 'config.py', 'main.py', 'frontend', 'feedback.json', 'history.json', and 'requirements.txt'. The 'schemas.py' file in the 'models' folder is selected. The main editor area shows the content of 'schemas.py', which defines several Pydantic models. The code is as follows:

```
1 #app/models/schemas.py
2 from pydantic import BaseModel
3
4 from typing import List
5
6 class EventInput (BaseModel):
7     description:str
8
9 class UserInterests (BaseModel):
10     interests: List[str]
11
12 class ConversationRequest (BaseModel):
13     description:str
14     interests: List[str]
15
16 class ConversationResponse (BaseModel):
17     topics: List[str]
18     suggestions: List[str]
19
20 class FactCheckRequest (BaseModel):
21     query: str
22
23 class FactCheckResponse (BaseModel):
24     summary: str
25
```

Event Analyzer Service

This service module is responsible for the intelligent theme extraction capability of the application. It uses the Hugging Face Transformers pipeline abstraction to load and run the DistilBERT model configured for zero-shot classification. At the module level, when the Python file is first imported, the zero-shot classification pipeline is immediately instantiated. This is an intentional design decision — loading the DistilBERT model into memory once at startup (rather than on every request) means that subsequent requests are processed much faster, since the expensive model-loading step only happens once during the application lifecycle. The `extract_event_themes` function accepts an event description string and an optional list of candidate labels. If no labels are provided, it defaults to a broad set of professional networking themes such as AI, healthcare, blockchain, education, and sustainability. The DistilBERT zero-shot classifier then scores each candidate label against the input text and returns the top three highest-scoring themes. These themes drive everything that follows — they become the context window for conversation generation.



```
app > services > event_analyzer.py > extract_event_themes
1 # app/services/event_analyzer.py
2
3 from transformers import pipeline
4 from app.config import MODEL_NAMES
5
6 classifier = pipeline(
7     "zero-shot-classification",
8     model=MODEL_NAMES["event_analysis"]
9 )
10
11 def extract_event_themes(description: str, candidate_labels=None):
12     if candidate_labels is None:
13         candidate_labels = [
14             "AI", "healthcare", "blockchain", "education", "sustainability", "finance", "cybersecurity", "marketing", "cloud"
15         ]
16
17     result = classifier(description, candidate_labels)
18
19     return result["labels"][:3]
```

Topic Generator Service

This service module handles the creative core of the application: generating natural, engaging conversation starters using GPT-2. Unlike the classifier which makes binary judgment calls, the generator produces original text that must sound human-like and contextually appropriate.

The `set_seed(42)` call at module level is a crucial detail for reproducibility. By fixing the random seed, the model will produce consistent outputs given the same input across different runs, which is invaluable for debugging and testing. The `generate_topics` function constructs a structured natural language prompt by interpolating the extracted event themes and user interests into a first-person context narrative. This prompt engineering approach guides GPT-2 toward producing responses in the desired format — conversation starters rather than arbitrary text.

The `max_length=80` parameter limits the generated text to approximately 80 tokens, ensuring suggestions remain concise and practical. After generation, the output is split by newline characters and the first three lines are extracted, cleaned of bullet markers, and returned as a list of strings. This post-processing step handles the fact that GPT-2 does not always produce cleanly formatted lists, requiring light cleanup to extract usable suggestions.

```
NETWORKING ASSISTANT  app > services > topic_generator.py > ...
> .pytest_cache
> .venv
> app
  > __pycache__
  > models
  > routers
  > services
    > __pycache__
    > event_analyzer.py
    > fact_checker.py
    > feedback_logger.py
    > history_logger.py
    > topic_generator.py
  > tests
  > config.py
  > main.py
  > frontend
  > venv
  {} feedback.json
  {} history.json
  {} requirements.txt

1 # app/services/topic_generator.py
2 from transformers import pipeline, set_seed
3 from app.config import MODEL_NAMES
4
5 generator = pipeline("text-generation", model=MODEL_NAMES["text_generator"])
6
7 set_seed(42)
8
9 def generate_topics(event_themes, user_interests):
10     prompt = (
11         f"I'm attending a networking event focused on {' '.join(event_themes)}. "
12         f"I'm personally interested in {' '.join(user_interests)}. "
13         f"What are three creative and engaging conversation starters I could use to break the ice? "
14         f"Return them as a bullet list."
15     )
16
17     outputs = generator(
18         prompt, max_length=80, num_return_sequences=1
19     )
20
21     suggestions = outputs[0]["generated_text"].split("\n")[:3]
22
23     return [
24         s.strip("- ").strip() for s in suggestions if s.strip()
25     ]
```

Fact Checker Service

This service provides the quick fact-verification capability by querying the Wikipedia API. The design philosophy here prioritizes simplicity and reliability — Wikipedia is a well-maintained, publicly accessible knowledge base that returns structured JSON data, making it ideal for this use case without requiring API keys or authentication.

The `fact_check` function takes a query string, constructs a URL to the Wikipedia REST API (which returns the first paragraph of the most relevant article in JSON format), and returns the text extract. The `try-except` block ensures that any network errors, timeout failures, or invalid JSON responses are caught gracefully, returning a user-friendly fallback message instead of crashing the application. This defensive programming approach is essential for maintaining application stability in production environments where external APIs can be unpredictable.

```
NETW...  app > services > fact_checker.py > fact_check
> .pytest_cache
> .venv
> app
  > __pycache__
  > models
  > routers
  > services
    > __pycache__
    > event_analyzer.py
    > fact_checker.py

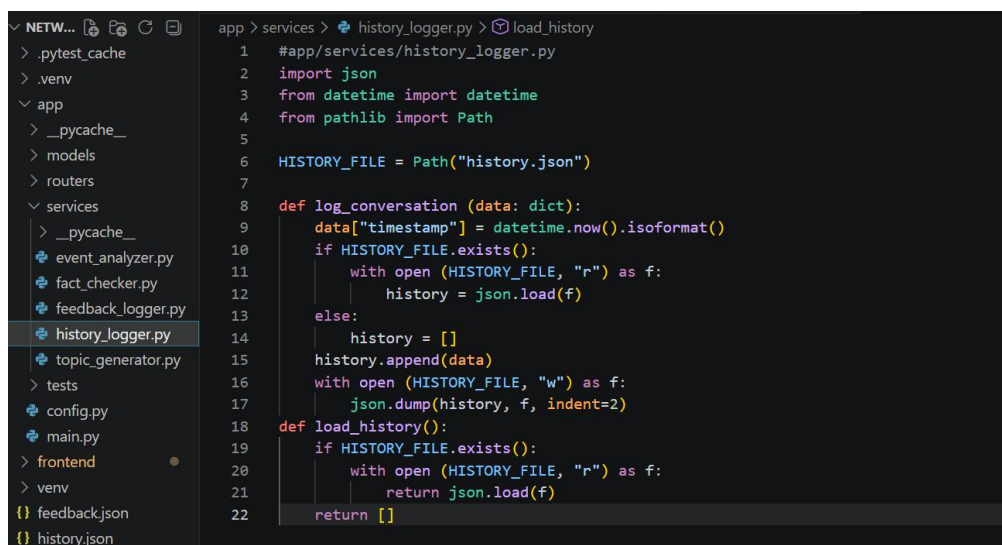
1 #app/services/fact_checker.py
2 import requests
3 from app.config import FACT_CHECK_API
4
5 def fact_check(query: str) -> str:
6     try:
7         response = requests.get(f"{FACT_CHECK_API}/{query}")
8         data = response.json()
9         return data.get("extract", "No summary found.")
10    except:
11        return "Fact-checking failed."
```

History Logger Service

The history logger provides persistent storage for conversation sessions. This enables the 'View Previous Conversations' feature in the frontend and allows users to revisit past interactions for reflection and learning.

The history logger operates on a simple append-to-JSON-list pattern. When `log_conversation` is called, it first adds an ISO-formatted timestamp to the data dictionary (ensuring we always know when each conversation was generated), then reads the existing history file if it exists (or initializes an empty list), appends the new entry, and writes the entire updated list back to disk. This read-modify-write pattern ensures data integrity for single-process use.

The use of Python's `pathlib.Path` for file handling rather than raw string paths is a best practice that ensures cross-platform compatibility — the same code works correctly on Windows, macOS, and Linux without modification. The `load_history` function provides a clean read interface that always returns a list, even when no history has been saved yet.



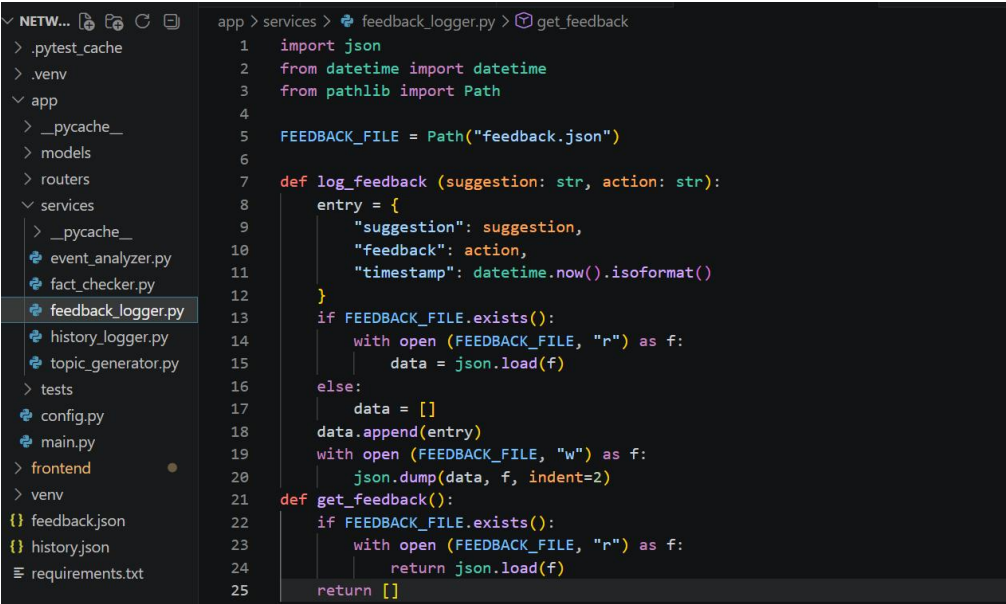
```
app > services > history_logger.py > load_history
1  #app/services/history_logger.py
2  import json
3  from datetime import datetime
4  from pathlib import Path
5
6  HISTORY_FILE = Path("history.json")
7
8  def log_conversation (data: dict):
9      data["timestamp"] = datetime.now().isoformat()
10     if HISTORY_FILE.exists():
11         with open (HISTORY_FILE, "r") as f:
12             history = json.load(f)
13     else:
14         history = []
15     history.append(data)
16     with open (HISTORY_FILE, "w") as f:
17         json.dump(history, f, indent=2)
18 def load_history():
19     if HISTORY_FILE.exists():
20         with open (HISTORY_FILE, "r") as f:
21             return json.load(f)
22     return []
```

Feedback Logger Service

The feedback logger follows the same architectural pattern as the history logger but serves a distinct purpose:

capturing explicit user feedback on individual conversation suggestions. This feedback data forms the foundation for a future recommendation improvement loop.

Each feedback entry captures three fields: the exact suggestion text that was rated (so we can correlate feedback with specific generated content), the action taken (either 'like' or 'dislike' corresponding to the thumbs-up and thumbs-down buttons in the UI), and a timestamp. This structured format makes it straightforward to perform analytics later — for example, identifying which types of suggestions consistently receive positive feedback versus neg



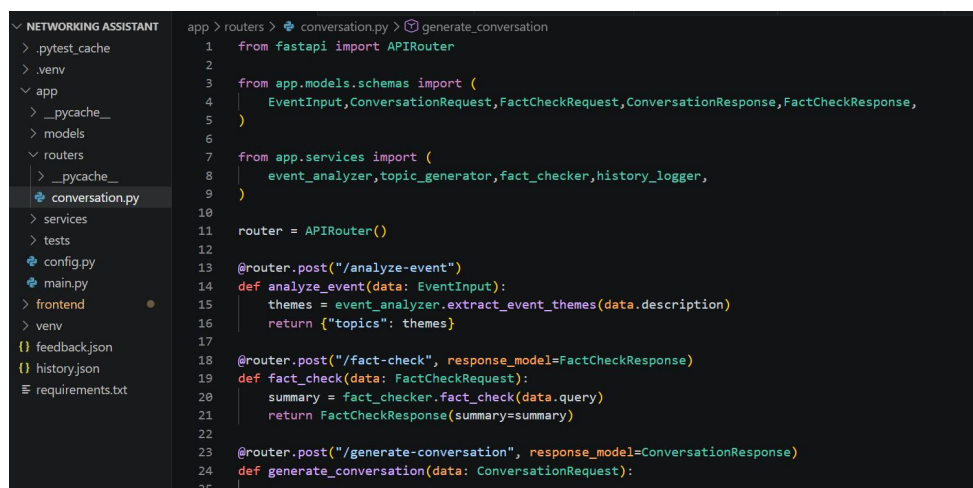
```
app > services > feedback_logger.py > get_feedback
1  import json
2  from datetime import datetime
3  from pathlib import Path
4
5  FEEDBACK_FILE = Path("feedback.json")
6
7  def log_feedback (suggestion: str, action: str):
8      entry = {
9          "suggestion": suggestion,
10         "feedback": action,
11         "timestamp": datetime.now().isoformat()
12     }
13     if FEEDBACK_FILE.exists():
14         with open (FEEDBACK_FILE, "r") as f:
15             data = json.load(f)
16     else:
17         data = []
18     data.append(entry)
19     with open (FEEDBACK_FILE, "w") as f:
20         json.dump(data, f, indent=2)
21     def get_feedback():
22         if FEEDBACK_FILE.exists():
23             with open (FEEDBACK_FILE, "r") as f:
24                 return json.load(f)
25     return []
```

API Routes

With all service modules built and tested in isolation, they are wired together through FastAPI's APIRouter in the [conversation.py](#) file. This routing layer acts as the integration point between the HTTP interface and the business logic layers, handling request deserialization, service orchestration, and response serialization.

The router defines three POST endpoints, each serving a distinct function. The `/analyze-event` endpoint provides a standalone theme extraction capability, useful for debugging or building custom integrations. The `/fact-check` endpoint wraps the Wikipedia service in a type-safe API contract. The `/generate-conversation` endpoint is the primary application endpoint that orchestrates the full pipeline: it calls the event analyzer to extract themes, passes those themes along with user interests to the topic generator, automatically saves the complete interaction to history, and returns both the topics and suggestions in a single structured response.

The key architectural insight here is that `/generate-conversation` performs automatic side-effect logging — every successful conversation generation is automatically persisted to history without requiring any additional action from the frontend. This separation of concerns means the frontend does not need to know about history logging; it simply makes a single API call and receives its response.



```
app > routers > conversation.py > generate_conversation
1  from fastapi import APIRouter
2
3  from app.models.schemas import (
4      EventInput, ConversationRequest, FactCheckRequest, ConversationResponse, FactCheckResponse,
5  )
6
7  from app.services import (
8      event_analyzer, topic_generator, fact_checker, history_logger,
9  )
10
11  router = APIRouter()
12
13  @router.post("/analyze-event")
14  def analyze_event(data: EventInput):
15      themes = event_analyzer.extract_event_themes(data.description)
16      return {"topics": themes}
17
18  @router.post("/fact-check", response_model=FactCheckResponse)
19  def fact_check(data: FactCheckRequest):
20      summary = fact_checker.fact_check(data.query)
21      return FactCheckResponse(summary=summary)
22
23  @router.post("/generate-conversation", response_model=ConversationResponse)
24  def generate_conversation(data: ConversationRequest):
25
```

```
✓ NETWORKING ASSISTANT  app > routers > conversation.py > generate_conversation
> .pytest_cache
> .venv
> app
> __pycache__
> models
> routers
> __pycache__
conversation.py
> services
> tests
config.py
main.py
frontend
> venv
{} feedback.json
{} history.json
≡ requirements.txt

24 def generate_conversation(data: ConversationRequest):
25
26     # Extract event themes
27     themes = event_analyzer.extract_event_themes(data.description)
28
29     # Generate conversation starters
30     suggestions = topic_generator.generate_topics(themes, data.interests)
31
32     # Save conversation history
33     history_logger.log_conversation({
34         "description": data.description,
35         "interests": data.interests,
36         "topics": themes,
37         "suggestions": suggestions
38     })
39
40     # Return response
41     return ConversationResponse(
42         topics=themes,
43         suggestions=suggestions
44     )
```

Application Entry Point

The [main.py](#) file is intentionally minimal, following the principle of doing one thing well. It creates the FastAPI application instance with a descriptive title (which appears in the auto-generated Swagger documentation), registers the conversation router (which brings in all three API endpoints), and defines a simple root health-check endpoint at GET /. This health-check endpoint is a production best practice — it allows load balancers, monitoring systems, and developers to quickly verify that the API is running and reachable before attempting any business-logic calls.

```
✓ NETW...  app > main.py > ...
> .pytest_cache
> .venv
> app
> __pycache__
> models
> routers
> services
> tests
config.py
main.py
frontend
> venv
{} feedback.json

1 #app/main.py
2 from fastapi import FastAPI
3 from app.routers import conversation
4 app = FastAPI (title="Personalized Networking Assistant")
5 app.include_router (conversation.router)
6 @app.get("/")
7 def root():
8     return {"message": "Welcome to the Networking Assistant API!"}
```

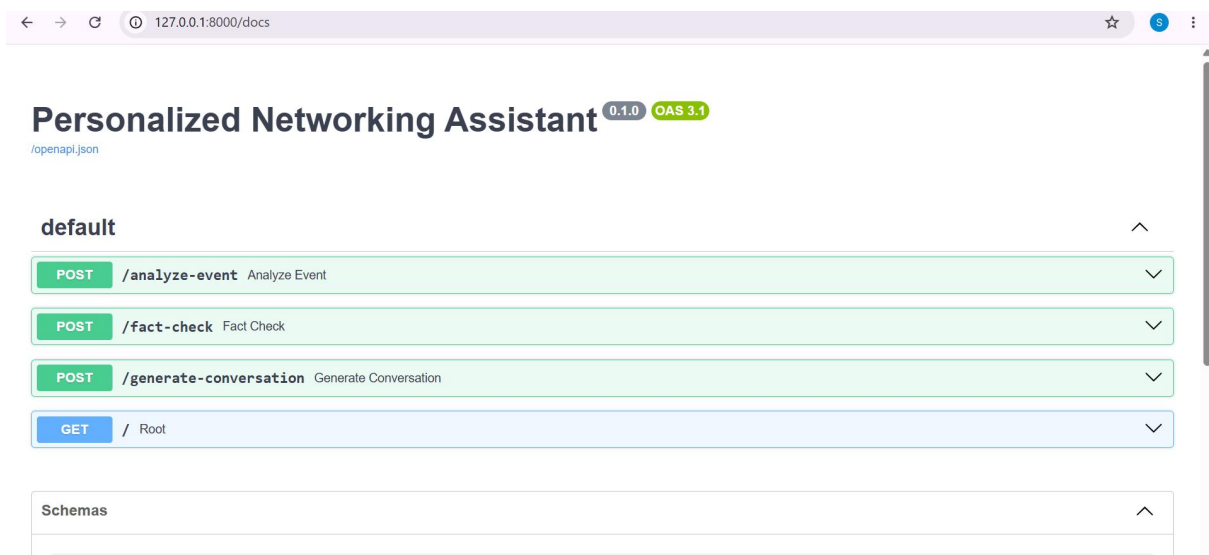
EPIC 3: MAIN APPLICATION LOGIC

The Centralized Routing Architecture

The routing architecture in this application follows a hub-and-spoke model where [main.py](#) acts as the central hub that assembles the application from modular components. This design pattern is especially important as applications grow — it allows new feature areas (such as a future `/users` endpoint or `/recommendations` endpoint) to be added as separate router files without touching existing code.

Understanding the Request Lifecycle

Understanding how a request flows through the system from the moment the frontend sends an HTTP POST to when the user sees a result is fundamental to understanding the application architecture:



Service Layer Design Principles

The service layer in this application is designed around several key software engineering principles that improve maintainability, testability, and extensibility:

Single Responsibility Principle

Each service file has exactly one job: event_[analyzer.py](#) only performs theme extraction, topic_[generator.py](#) only generates text, and so on. This means that if the fact-checking mechanism needs to change (e.g., switching from Wikipedia to a different API), only fact_[checker.py](#) needs to be modified — all other code remains untouched

Dependency Injection via Imports

Services are imported into the router at the top of [conversation.py](#), making the dependencies explicit and easy to identify. In testing scenarios, these imports can be mocked to test route logic in isolation from the actual AI models, which is important because loading DistilBERT and GPT-2 in every test run would make tests prohibitively slow.

Stateless Service Functions

All service functions are stateless — they take input parameters and return output without modifying any shared state. The only exceptions are the logger functions, which intentionally maintain state in JSON files. This design makes service functions trivially testable: given the same inputs, they always produce deterministic outputs (for the loggers) or model-dependent outputs (for the AI services).

FastAPI-Specific Features Leveraged

The application takes advantage of several FastAPI-specific features that make the API more robust, self-documenting, and developer-friendly:

Automatic OpenAPI Documentation Because all endpoints use Pydantic models for both request and response bodies, FastAPI automatically generates a complete OpenAPI 3.0 specification and serves it through a built-in Swagger UI at <http://127.0.0.1:8000/docs>. This means developers can

test all endpoints interactively through the browser without needing any additional tools like Postman or curl.

Type-Safe Request Validation When a request arrives at any endpoint, FastAPI automatically validates the JSON body against the corresponding Pydantic schema before the handler function is even called. If the description field is missing from a ConversationRequest, or if interests is sent as a string instead of a list, FastAPI returns a detailed 422 error explaining exactly what was wrong — with no additional validation code required from the developer.

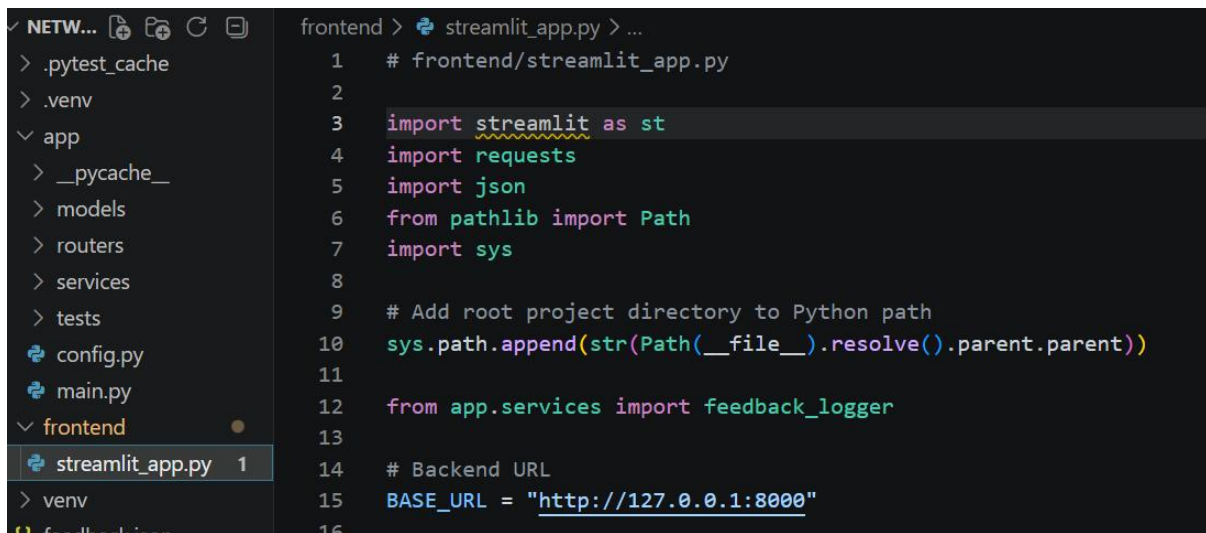
Response Model Enforcement The response_model parameter on endpoint decorators (e.g., response_model=ConversationResponse) ensures that the data returned from the handler function is validated and serialized correctly before being sent to the client. This prevents accidentally returning internal data structures or sensitive information that was not intended to be part of the API response.

Epic 4: Frontend UI Using Streamlit

Application Setup and Configuration

The Streamlit frontend begins with its imports and backend connection configuration. This section establishes the bridge between the Python frontend and the FastAPI backend:

The sys.path.append line is a critical piece of Python path manipulation that allows the Streamlit script (located in the frontend/ directory) to import modules from the parent project directory (specifically the feedback_logger service). Without this, Python's module system would not be able to locate the [app.services](#) package from the frontend's working directory. The BASE_URL constant defines the backend API location, which can easily be changed to a remote URL when deploying to a cloud environment.



```
frontend > streamlit_app.py > ...
1  # frontend/streamlit_app.py
2
3  import streamlit as st
4  import requests
5  import json
6  from pathlib import Path
7  import sys
8
9  # Add root project directory to Python path
10 sys.path.append(str(Path(__file__).resolve().parent.parent))
11
12 from app.services import feedback_logger
13
14 # Backend URL
15 BASE_URL = "http://127.0.0.1:8000"
16
```

Input Section and Main Generation Flow

The input section is the first thing users interact with when they open the application. It collects the two pieces of information needed to generate personalized conversation starters:

Streamlit's programming model is fundamentally different from traditional web frameworks: the entire Python script runs from top to bottom every time the user interacts with the UI. This means that if we stored results in regular Python variables, they would be lost the moment the user clicked another button. The `st.session_state` dictionary solves this problem by persisting data across re-runs within the same browser session.

The interest parsing logic `[i.strip() for i in user_interests.split(',')]` is a compact but important piece of data cleaning. When users type 'AI, blockchain, healthcare', the split produces `['AI', ' blockchain', ' healthcare']` — with leading spaces. The `strip()` call ensures these are cleaned to `['AI', 'blockchain', 'healthcare']` before being sent to the backend, preventing subtle matching issues in the AI models.

```
NETWORKING ASSISTANT
> .pytest_cache
> .venv
> app
> _pycache_
> models
> routers
> services
> tests
> config.py
> main.py
> frontend
  streamlit_app.py 1
> venv
() feedback.json
() history.json
≡ requirements.txt

frontend > streamlit_app.py > ...
12 from app.services import feedback_logger
13
14 # Backend URL
15 BASE_URL = "http://127.0.0.1:8000"
16
17 # Title
18 st.title("💖 Personalized Networking Assistant")
19 st.markdown(
20     "Generate conversation starters for networking events based on your interests."
21 )
22
23 # Input Section
24 event_description = st.text_area("📝 Enter Event Description")
25 user_interests = st.text_input("🏷️ Your Interests (comma-separated)")
26
27 if st.button("Generate Conversation Starters"):
28     if event_description and user_interests:
29
30         payload = {
31             "description": event_description,
32             "interests": [
33                 i.strip()
34                 for i in user_interests.split(",")
35             ]
36         }
37
38         response = requests.post(
39             f"{BASE_URL}/generate-conversation",
40             json=payload
41         )
42
43         if response.status_code == 200:
44             data = response.json()
45
46             # Store results
47             st.session_state["topics"] = data["topics"]
48             st.session_state["suggestions"] = data["suggestions"]
49
50         else:
51             st.error("Failed to generate conversation starters.")
52
53     else:
54         st.warning("Please enter both description and interests.")
55
```

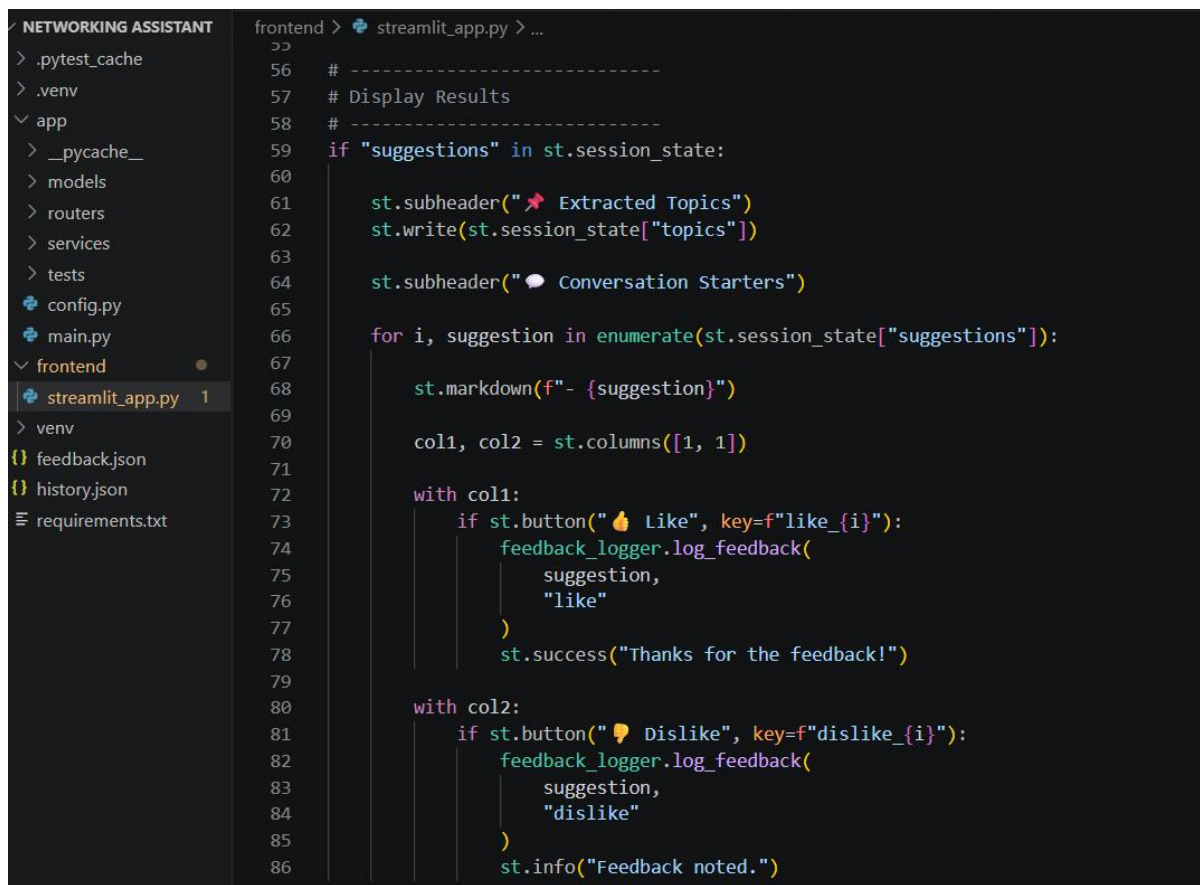
Results Display and Feedback System

After generation, the results are displayed and the feedback system is presented to the user. This section demonstrates Streamlit's ability to create dynamic, responsive layouts:

The conditional check 'if suggestions in st.session_state' is fundamental to Streamlit UI design. Since the entire script re-runs on every interaction, this guard prevents the results section from rendering before the user has clicked the generate button. Only when suggestions have been stored in session_state will this section appear, creating an experience of dynamic content that appears only when relevant.

The use of `st.columns([1, 1])` creates a two-column layout for the thumbs-up and thumbs-down buttons. This is a Streamlit layout primitive that divides the available horizontal space equally between two columns. The `key=f'like_{i}'` and `key=f'dislike_{i}'` parameters are critical: without

unique keys, Streamlit cannot distinguish between buttons in a loop and may trigger incorrect callbacks. By including the index *i* in the key, each button has a globally unique identifier within the session.

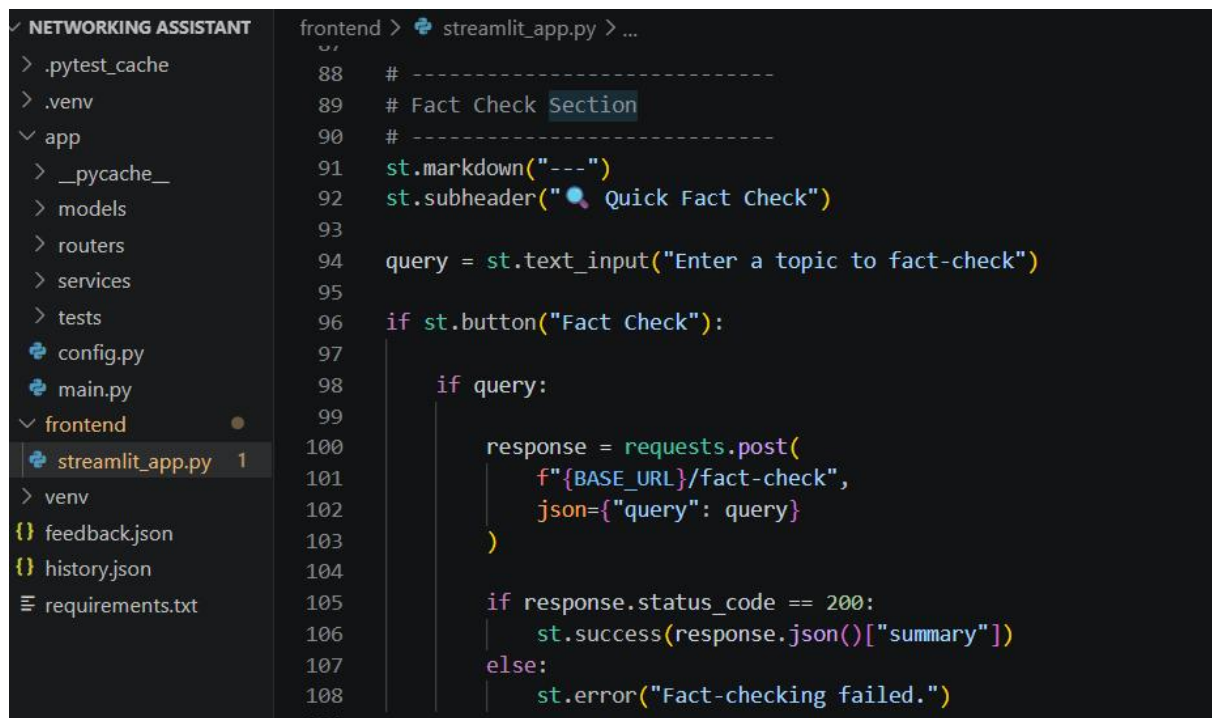


```
frontend > streamlit_app.py > ...
56 # -----
57 # Display Results
58 # -----
59 if "suggestions" in st.session_state:
60
61     st.subheader("🔍 Extracted Topics")
62     st.write(st.session_state["topics"])
63
64     st.subheader("💬 Conversation Starters")
65
66     for i, suggestion in enumerate(st.session_state["suggestions"]):
67
68         st.markdown(f"--- {suggestion}")
69
70         col1, col2 = st.columns([1, 1])
71
72         with col1:
73             if st.button("👍 Like", key=f"like_{i}"):
74                 feedback_logger.log_feedback(
75                     suggestion,
76                     "like"
77                 )
78                 st.success("Thanks for the feedback!")
79
80         with col2:
81             if st.button("👎 Dislike", key=f"dislike_{i}"):
82                 feedback_logger.log_feedback(
83                     suggestion,
84                     "dislike"
85                 )
86                 st.info("Feedback noted.")
```

Fact-Checking Section

The fact-checking section provides a completely independent feature within the same Streamlit page. Users can verify information about any topic without needing to re-enter their event description:

The `st.markdown('---')` call renders a horizontal divider line, creating a clear visual separation between the conversation generation section and the fact-checking section. This UX pattern helps users quickly understand that these are two distinct features on the same page. The `st.success()` function renders the fact-check result in a green highlighted box, which provides immediate positive visual feedback and draws the user's attention to the result.



```
frontend > streamlit_app.py > ...
88 # -----
89 # Fact Check Section
90 # -----
91 st.markdown("---")
92 st.subheader("🔍 Quick Fact Check")
93
94 query = st.text_input("Enter a topic to fact-check")
95
96 if st.button("Fact Check"):
97     if query:
98
99         response = requests.post(
100             f"{BASE_URL}/fact-check",
101             json={"query": query}
102         )
103
104         if response.status_code == 200:
105             st.success(response.json()["summary"])
106         else:
107             st.error("Fact-checking failed.")
108
109
```

Conversation History View

The history section allows users to review past conversations. This leverages the history.json file maintained by the backend's history_logger service:

The history[-5:] slice retrieves only the five most recent entries from the full history list, preventing the UI from becoming overwhelming if many sessions have been logged. The reversed() call then displays them in reverse chronological order (newest first), which aligns with the natural expectation that the most recent conversations should appear at the top. Each history entry is rendered with clear labels and a horizontal divider, making individual sessions easy to scan and distinguish.

```
NETW... [Icons]
> .pytest_cache
> .venv
> app
  > _pycache_
  > models
  > routers
  > services
  > tests
  > config.py
  > main.py
  > frontend
    > streamlit_app.py 1
  > venv
  {} feedback.json
  {} history.json
  requirements.txt
OUTLINE

frontend > streamlit_app.py > ...
110 # -----
111 # Conversation History
112 # -----
113 st.markdown("---")
114 st.subheader("📄 View Previous Conversations")
115
116 if st.button("Show History"):
117
118     history_path = Path("history.json")
119
120     if history_path.exists():
121         with open(history_path, "r") as f:
122             history = json.load(f)
123             for item in reversed(history[-5:]):
124                 st.markdown(f"***{item['timestamp']}***")
125                 st.write("***Event:**", item["description"])
126                 st.write(
127                     "***Interests:**",
128                     ", ".join(item["interests"])
129                 )
130                 st.write(
131                     "***Topics:**",
132                     ", ".join(item["topics"])
133                 )
134                 st.write("***Suggestions:**")
135                 for s in item["suggestions"]:
136                     st.markdown(f"- {s}")
137             st.markdown("---")
138     else:
139         st.info("No history found yet.")
140
141 # Feedback History
142 st.markdown("---")
143 st.subheader("📄 View Feedback History")
```

Feedback History View

The feedback history section allows users to review which suggestions they previously rated, creating a feedback loop for self-reflection:

The feedback view shows up to 10 recent feedback entries (double the history limit, since individual suggestion ratings are more granular than full conversation sessions). The ternary expression for the icon variable — using a thumbs-up emoji for 'like' and thumbs-down for 'dislike' — provides an immediately recognizable visual indicator of the feedback sentiment. The `st.caption()` function renders the timestamp in a smaller, subdued font, creating a clear visual hierarchy where the suggestion content is prominent and the metadata is secondary.



```
140
141 # Feedback History
142 st.markdown("---")
143 st.subheader("📄 View Feedback History")
144
145 feedback_path = Path("feedback.json")
146
147 if st.button("Show Feedback"):
148
149     if feedback_path.exists():
150
151         with open(feedback_path, "r") as f:
152             feedback_data = json.load(f)
153
154         for item in reversed(feedback_data[-10:]):
155
156             icon = "👍" if item["feedback"] == "like" else "👎"
157
158             st.markdown(
159                 f"{icon} **{item['suggestion']}**"
160             )
161
162             st.caption(f"🕒 {item['timestamp']}")
163             st.markdown("---")
164
165     else:
166         st.info("No feedback submitted yet.")
```

Streamlit State Management Philosophy

the entire Python script on every user interaction. The `st.session_state` dictionary acts as a persistent storage mechanism that preserves variables, user inputs, and application data across reruns, enabling developers to maintain conversational context, track user actions, and build dynamic interactive applications without losing state information.

Epic 5: Testing and Local Deployment

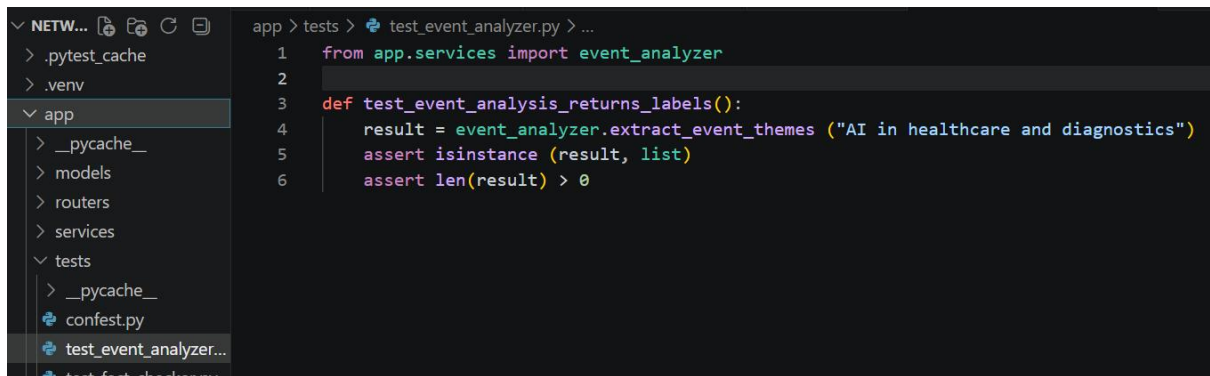
Testing Philosophy and Framework Selection

The project uses `pytest` as its testing framework, which is the de-facto standard for Python testing. For testing FastAPI endpoints specifically, the `httpx` library's `TestClient` is used, which allows API calls to be made in-process without needing a running server. This makes tests fast, reliable, and completely self-contained.

Testing the Event Analyzer Service

Unit tests for the event analyzer verify that the DistilBERT classification pipeline behaves correctly across a range of inputs:

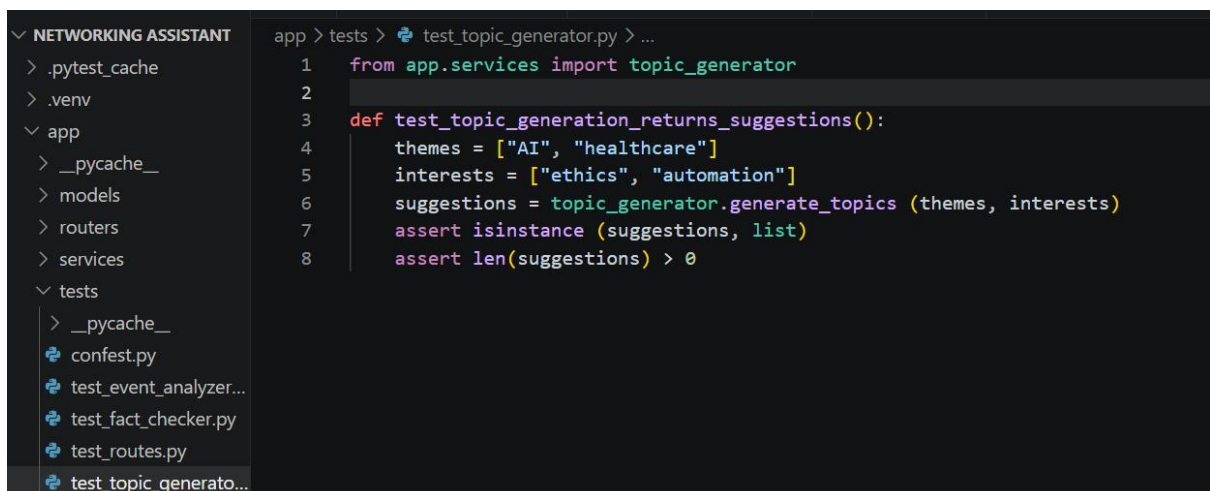
These tests validate the function's contract without making assertions about the specific themes returned (which would be brittle since the model's output depends on its training). Instead, they test structural properties: is the output a list? Does it contain at most three items? Are all items drawn from the candidate labels set? Does it return at least one result? These kinds of assertions remain valid regardless of the specific model weights or future model updates



```
app > tests > test_event_analyzer.py > ...
1  from app.services import event_analyzer
2
3  def test_event_analysis_returns_labels():
4      result = event_analyzer.extract_event_themes ("AI in healthcare and diagnostics")
5      assert isinstance (result, list)
6      assert len(result) > 0
```

Testing the Topic Generator Service

The topic generator tests follow a similar pattern, validating structure rather than specific content: The `test_generate_strings` and `test_generate_non_empty_strings` tests are particularly important because they validate the post-processing logic in `generate_topics`. The function strips bullet markers and whitespace from GPT-2's raw output — these tests verify that this cleanup produces actual string content and not accidentally empty strings, which would render as blank bullet points in the UI.

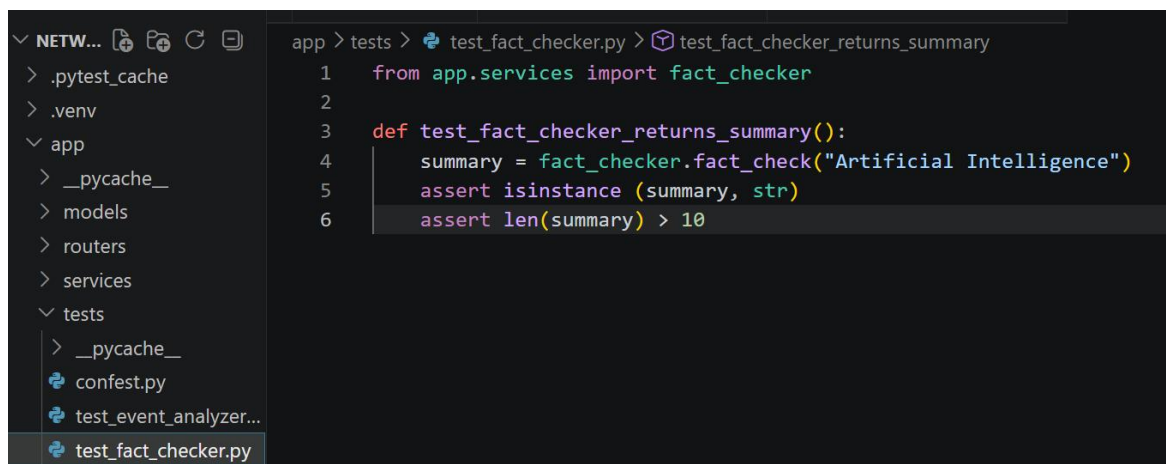


```
app > tests > test_topic_generator.py > ...
1  from app.services import topic_generator
2
3  def test_topic_generation_returns_suggestions():
4      themes = ["AI", "healthcare"]
5      interests = ["ethics", "automation"]
6      suggestions = topic_generator.generate_topics (themes, interests)
7      assert isinstance (suggestions, list)
8      assert len(suggestions) > 0
```

Testing the Fact Checker Service

Fact checker tests require a different strategy because they involve an external network call to the Wikipedia API. In true unit testing philosophy, external dependencies should be mocked to isolate the code under test:

These tests use Python's `unittest.mock` module to replace the actual `requests.get` function with a `MagicMock` object that returns controlled, predictable data. This eliminates the dependency on network connectivity and Wikipedia's availability during test runs, making the tests reliable in any environment including CI/CD pipelines that may not have internet access. The three test cases cover the happy path, the missing-data path, and the error path, ensuring all branches of the `fact_check` function are tested.

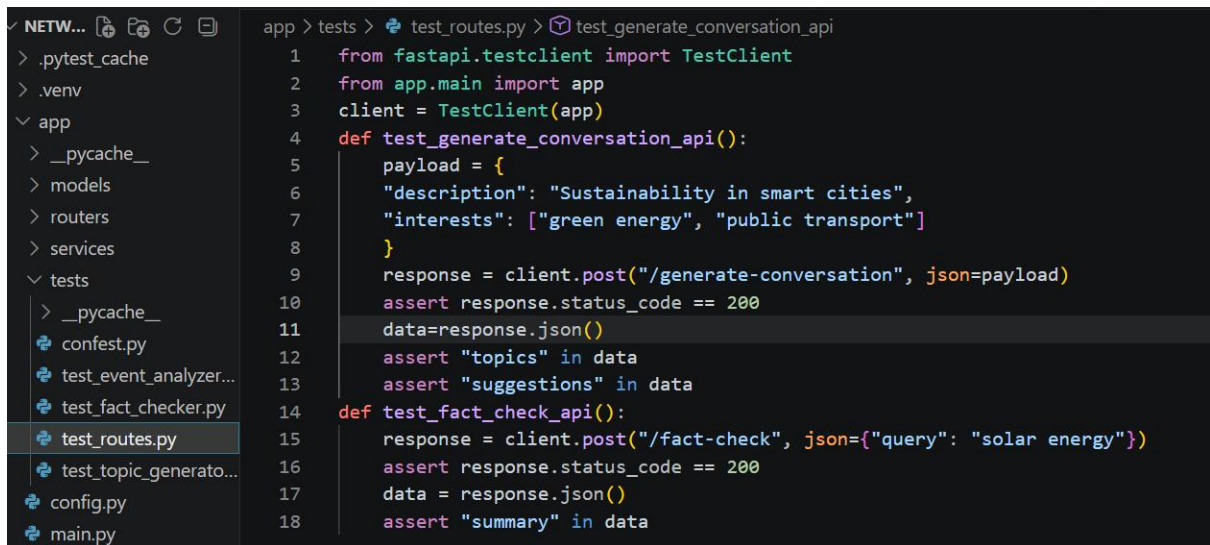


```
app > tests > test_fact_checker.py > test_fact_checker_returns_summary
1  from app.services import fact_checker
2
3  def test_fact_checker_returns_summary():
4      summary = fact_checker.fact_check("Artificial Intelligence")
5      assert isinstance(summary, str)
6      assert len(summary) > 10
```

Testing API Routes with httpx TestClient

Integration tests for the API routes validate that the complete request-response cycle works correctly, including request validation, service orchestration, and response serialization: The `TestClient` wraps the FastAPI application and allows tests to make HTTP requests directly to the application without network overhead. The `test_invalid_request_returns_422` test is particularly valuable — it verifies that FastAPI's automatic validation is working correctly by confirming that sending an empty payload to an endpoint that requires a description field results in the expected

422 Unprocessable Entity status code. This tests the Pydantic validation layer without writing any custom validation code.

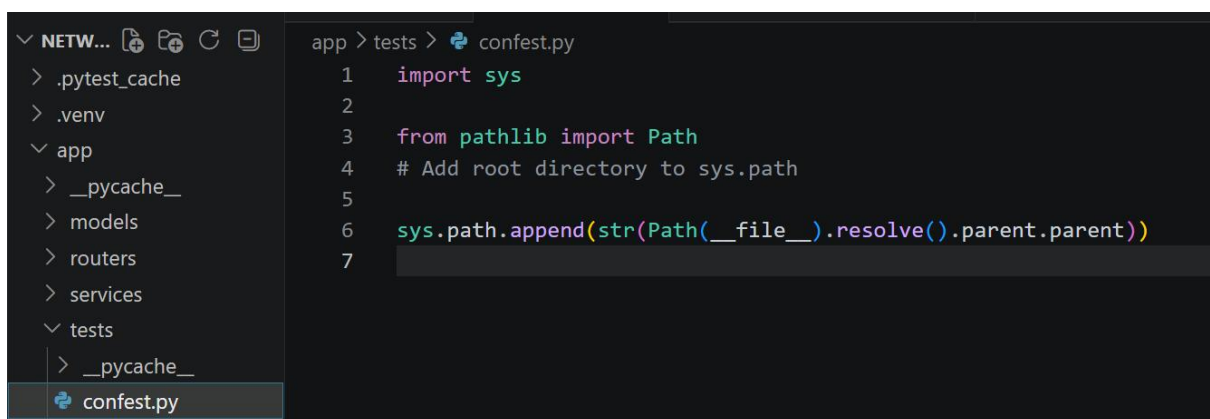


```
app > tests > test_routes.py > test_generate_conversation_api
1 from fastapi.testclient import TestClient
2 from app.main import app
3 client = TestClient(app)
4 def test_generate_conversation_api():
5     payload = {
6         "description": "Sustainability in smart cities",
7         "interests": ["green energy", "public transport"]
8     }
9     response = client.post("/generate-conversation", json=payload)
10    assert response.status_code == 200
11    data=response.json()
12    assert "topics" in data
13    assert "suggestions" in data
14 def test_fact_check_api():
15     response = client.post("/fact-check", json={"query": "solar energy"})
16     assert response.status_code == 200
17     data = response.json()
18     assert "summary" in data
```

Running Tests and Interpreting Results

Tests are executed using pytest from the project root directory. The framework automatically discovers all test files matching the test_*.py naming pattern and executes every function beginning with test_:

The --cov flag generates a code coverage report showing which lines of the application code were executed during tests and which were not. A target coverage of 80%+ is considered healthy for a production application. Lines not covered by tests represent potential blind spots where bugs could lurk undetected.



```
app > tests > confest.py
1 import sys
2
3 from pathlib import Path
4 # Add root directory to sys.path
5
6 sys.path.append(str(Path(__file__).resolve().parent.parent))
7
```

Local Deployment

With all tests passing, the application is deployed locally for full end-to-end manual testing. The system requires two concurrent processes: the FastAPI backend server and the Streamlit frontend server:

The `--reload` flag on the `uvicorn` command enables hot-reloading: the server automatically restarts whenever a Python file in the project is modified. This dramatically accelerates the development feedback loop since code changes take effect immediately without manual server restarts. Once both servers are running, the complete application is accessible at <http://localhost:8501> for the Streamlit UI and <http://127.0.0.1:8000/docs> for the interactive API documentation.

Manual Testing and Verification

Beyond automated tests, manual testing through the Streamlit UI provides validation of the complete end-to-end user experience.

Written for event_analyzer, topic_generator, fact_checker, and all routes

Used pytest and httpx TestClient for FastAPI Activity 5.2: Deployment on Localhost

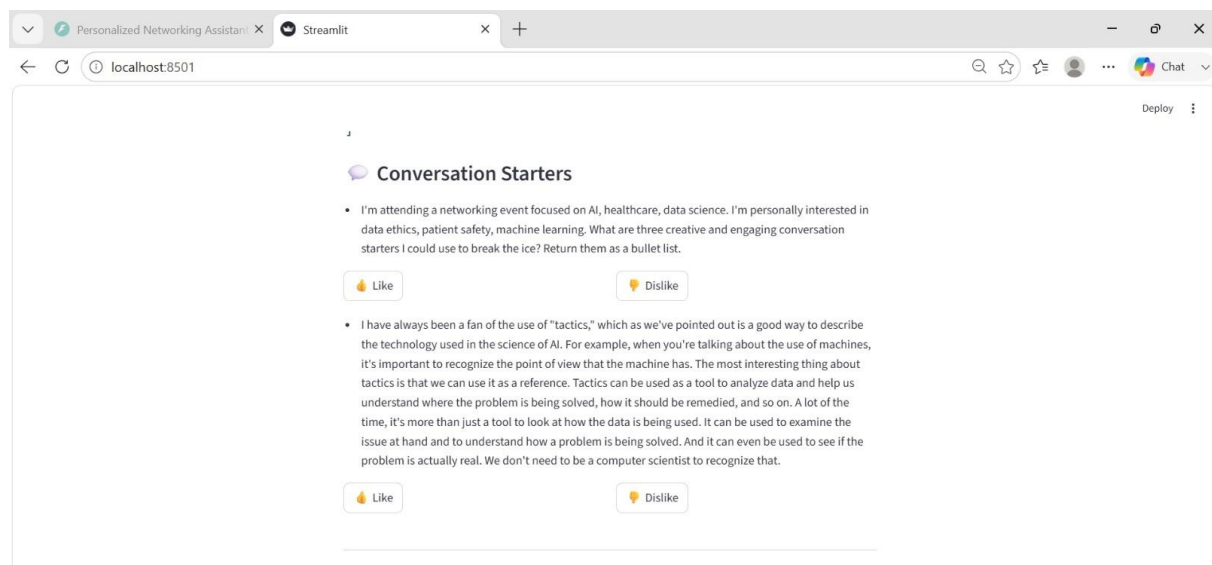
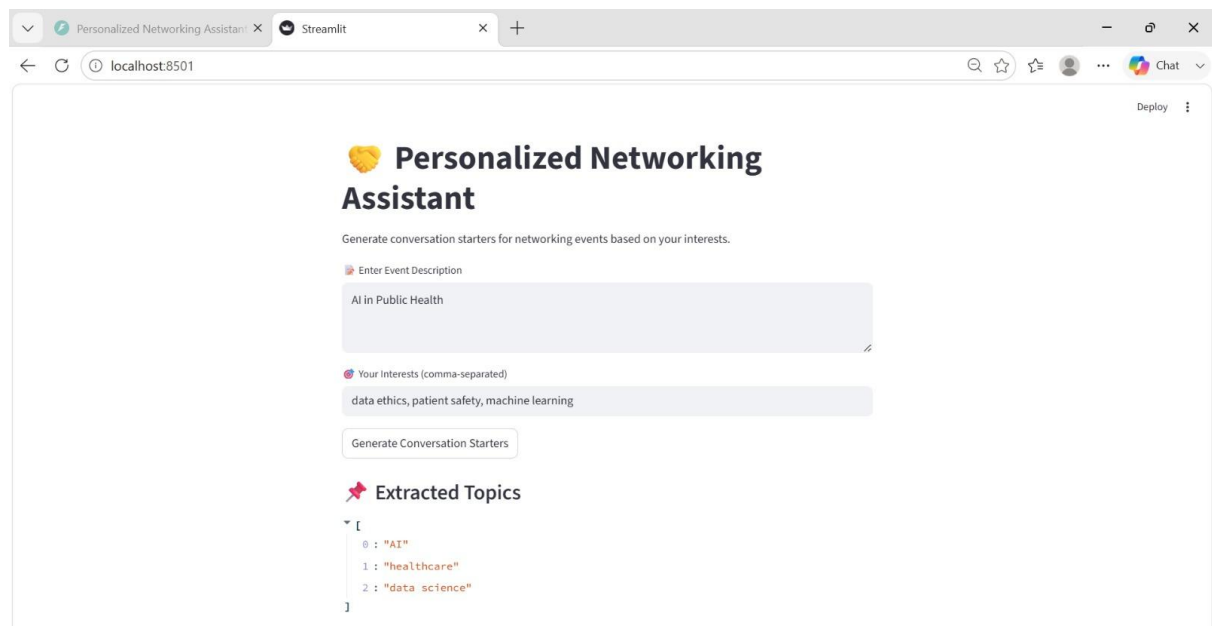
Streamlit app: <http://localhost:8501>

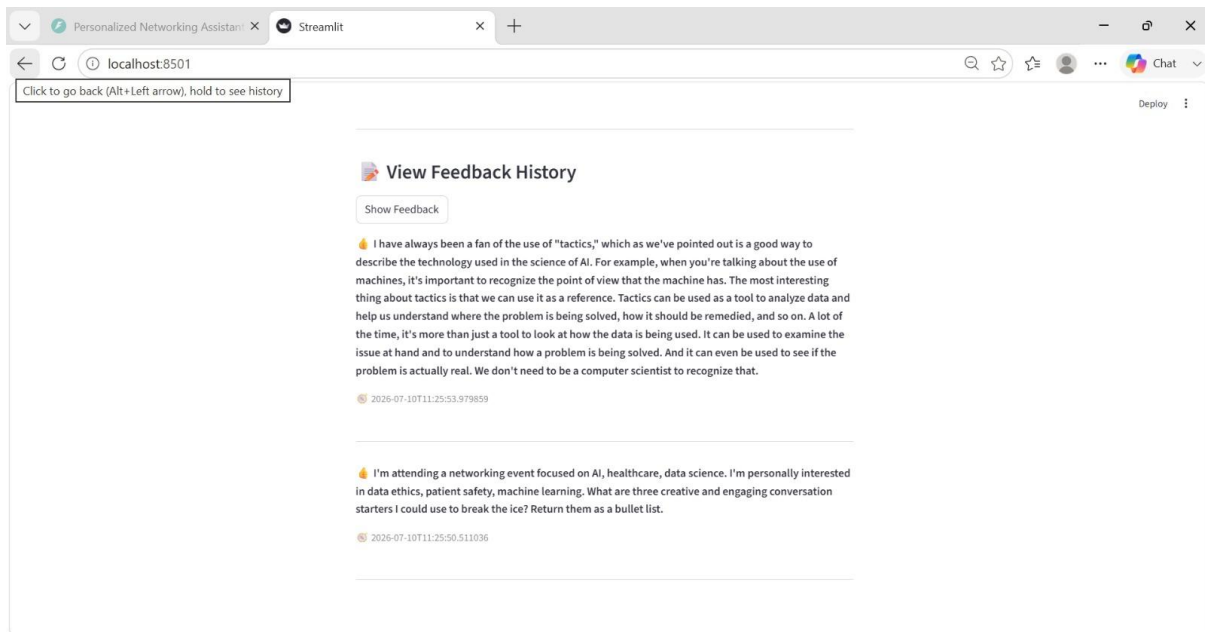
FastAPI Swagger UI: <http://127.0.0.1:8000/docs>

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

```
t\.\venv\Scripts\Activate.ps1")  
• (.env) PS D:\networking assistant> .\venv\Scripts\Activate  
❖ (venv) PS D:\networking assistant> uvicorn app.main:app --reload  
INFO: Will watch for changes in these directories: ['D:\\networking assistant']  
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)  
INFO: Started reloader process [24636] using StatReload  
  
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.  
Loading weights: 100% |██████████████████████████████████████████████████| 515/515 [00:00<00:00, 1430.78it/s]  
Loading weights: 100% |███████████████████████████████████████████████████| 148/148 [00:00<00:00, 1339.63it/s]  
  
INFO: Started server process [25208]  
INFO: Waiting for application startup.  
INFO: Application startup complete.  
INFO: 127.0.0.1:56650 - "GET /docs HTTP/1.1" 200 OK  
INFO: 127.0.0.1:56650 - "GET /openapi.json HTTP/1.1" 200 OK
```

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS D:\networking assistant> streamlit run frontend/streamlit_app.py
Local URL: http://localhost:8501
Network URL: http://192.168.1.34:8501
█
```





Conclusion:

The Personalized Networking Assistant represents a complete and successful implementation of an AI-powered networking support system designed to enhance professional communication and user confidence during networking events. The project was developed through a structured and systematic approach divided into five major Epics, beginning with research and model evaluation, followed by system architecture design, backend API development, frontend interface creation, integration of AI services, and finally testing and deployment. This step-by-step methodology ensured that every component of the application was carefully planned, implemented, and validated before moving to the next stage of development.

One of the major strengths of the project lies in the selection of technologies and frameworks that balance performance, usability, and deployment feasibility. **DistilBERT** was chosen as the primary zero-shot classification model due to its lightweight architecture, fast inference speed, and reliable contextual understanding capabilities. **GPT-2 Small** was integrated for conversation starter generation because it provides efficient natural language generation while remaining suitable for systems with limited computational resources. **FastAPI** was selected for backend development because of its high performance, asynchronous capabilities, and strong support for API

documentation and type validation. **Streamlit** enabled rapid frontend development and provided an intuitive user interface that supports real-time interaction with minimal complexity.

The application successfully demonstrates several practical and innovative features. First, it generates intelligent and personalized networking conversation starters based on the user's interests, professional background, and event context. This allows users to initiate conversations more naturally and confidently. Second, the integrated Wikipedia-based fact-checking system provides users with quick topic verification and contextual knowledge support, reducing uncertainty while discussing unfamiliar subjects. Third, the inclusion of feedback collection and conversation history tracking enables users to evaluate previous interactions and continuously improve their networking strategies over time.

Another important achievement of the project is the successful integration of multiple technologies into a cohesive full-stack AI system. The frontend, backend APIs, machine learning models, and external knowledge verification services communicate efficiently, demonstrating strong modular architecture and scalability potential. Furthermore, the application was designed to operate in a locally deployable environment, making it accessible for educational, research, and small-scale professional use without requiring expensive cloud infrastructure.

Overall, the Personalized Networking Assistant showcases the practical application of Natural Language Processing, machine learning, and modern web technologies in solving real-world communication challenges. The project not only fulfills its functional objectives but also establishes a strong foundation for future enhancements such as multilingual support, advanced recommendation systems, speech-based interaction, and cloud deployment capabilities.